Programming Assignment 2 - Introduction to Game Theory

Spring 2026

Kindly note the following instructions before starting the assignment.

- The following assignment aims to enhance your understanding of the theory and implementation of auctions.
- This is a group assignment.
- You can choose to attempt Questions 1 and 2, or alternatively choose any one setting from Question 3.
- All questions carry equal weightage of 10 marks.
- If you are choosing from any setting from Question 3, or wish to implement any other algorithmic result on auctions, kindly email the paper (along with page number) as a reference before proceeding.
- Kindly refer to Algorithmic Game Theory (AGT) by Noam Nisan, Tim Roughgarden, Eva Tardos, Vijay V. Vazirani for any doubts, and some of the papers(pg 296) for Question 3.
- Note that you can choose any programming language to implement.
- However, you are not supposed to use any library that directly implements the algorithm.
- You are required to submit a *teamname*.zip file that contains modular code, including, but not limited to:
  - *main*: This takes the input, which is typically two lines, where the first line is the number of items n, and the next line is the number of bidders m.
  - *initialize*: This initializes the prices, valuations, or demands (depending on which question you are implementing).
    - You can initialize randomly, while satisfying the constraints of the algorithm, to test your code.
    - **Important**: Add comments to describe how you are taking the input valuations/prices/demands, etc.

- *algo*: This implements the main algorithm.
- *log:* This is a text file containing the information, including:
  1. The initialized bids, prices, and demands
  2. The winners, or losers, after each step
  3. Intermediate allocation (if any)
  4. The final allocation and its welfare
  5. Any other important intermediate values, used to decide the outcomes of the next step.
  6. Here's a sample log

     Number of bidders: 10
     Number of items: 2
     Bidder 1 Valuation: {"1":15,"2":20,"12":30}
     Bidder 2 Valuation: {"1":10,"2":25,"12":30}
     *** Iteration 1 ***
     Items: [1,2]
     Prices: [0,0]
     Bidders Demands
     Bidder 1: {"12"}, Bidder 2: {"12"}
     Allocation $S_1 = \emptyset, S_2 = \emptyset$
     *** Iteration 2 ***
     Items: [1,2]
     Prices: [10,10]
     Bidders Demands
     Bidder 1: {"12"}, Bidder 2: {"12"}
     Allocation $S_1 = \emptyset, S_2 = \emptyset$
     *** Iteration 3 ***
     Items: [1,2]
     Prices: [20,20]
     Bidders Demands
     Bidder 1: {"2"}, Bidder 2: {"2"}
     Allocation $S_1 = \emptyset, S_2 = \emptyset$
     *** Iteration 4 ***
     Items: [1,2]
     Prices: [25,25]
     Bidders Demands
     Bidder 1: $\emptyset$, Bidder 2: {"2"}
     Allocation $S_1 = \emptyset, S_2 = \{2\}$
     *** Iteration 5 ***
     Items: [1]

Prices: [20]
Bidders Demands
Bidder 1: $\emptyset$, Bidder 2: $\emptyset$
Allocation $S_1 = \emptyset, S_2 = \{2\}$
*** Iteration 5 ***
Items: [1]
Prices: [15]
Bidders Demands
Bidder 1: $\{1\}$, Bidder 2: $\emptyset$
Allocation $S_1 = \{1\} S_2 = \{2\}$

Final Allocation: $S_1 = \{1\} S_2 = \{2\}$
Welfare: 40

- Note that you are expected to write a clear code, along with comments (if you are implementing any non-trivial optimization within your code).
- Your code will be tested by modifying the *initialization* code and by allowing custom demands/bids/prices as per the algorithm.

1 Ascending Auctions

The auctioneer publishes prices, initially set to zero (or some other minimum prices), and the bidders repeatedly respond to the current prices by bidding on their most desired bundle of goods under the current prices. The auctioneer then repeatedly updates the prices by increasing some of them, until a set of prices is reached at which the auctioneer can declare an allocation.

Definition 1 (Substitutes (Gross Substitutes)) *A valuation v_i satisfies the* substitutes *(or* gross-substitutes*) property if for every pair of item-price vectors $\vec{q} \geq \vec{p}$ (coordinate-wise), the demand at prices $\vec{q}$ contains all items in the demand at prices $\vec{p}$ whose price remained constant.*

Formally, for every

$$A \in \arg\max_S \left\{ v(S) - \sum_{j \in S} p_j \right\},$$

there exists

$$D \in \arg\max_S \left\{ v(S) - \sum_{j \in S} q_j \right\}$$

such that

$$D \supseteq \{j \in A \mid p_j = q_j\}.$$

An item-price ascending auction for substitutes valuations:

Initialization:
For every item $j \in M$, set $p_j \leftarrow 0$.
For every bidder i let $S_i \leftarrow \emptyset$.

Repeat
For each i, let D_i be the demand of i at the following prices:
p_j for $j \in S_i$ and $p_j + \epsilon$ for $j \notin S_i$.
If for all i, $S_i = D_i$, exit the loop;
Find a bidder i with $S_i \neq D_i$ and update:

- For every item $j \in D_i \setminus S_i$, set $p_j \leftarrow p_j + \epsilon$
- $S_i \leftarrow D_i$
- For every bidder $k \neq i$, $S_k \leftarrow S_k \setminus D_i$

Finally: Output the allocation $S_1, \ldots, S_n$.

2 Ascending Bundle-Price Auctions

This auction involves a more complex pricing scheme which allows personalized bundle prices - a distinct price for each possible bundle and for each bidder. That is, personalized bundle prices specify a price $p_i(S)$ per each bundle S.

A bundle price auction:

Initialization: For every player i and bundle S, let $p_i(S) \leftarrow 0$.

Repeat

- Find an allocation $T_1, \ldots, T_n$ that maximizes revenue at current prices, i.e., $\sum_{i=1}^{n} p_i(T_i) \geq \sum_{i=1}^{n} p_i(Y_i)$ for any other allocation $Y_1, \ldots, Y_n$.
  (Bundles with zero prices will not be allocated, i.e., $p_i(T_i) > 0$ for every i.)
- Let L be the set of losing bidders, i.e., $L = \{i \mid T_i = \emptyset\}$.
- For every $i \in L$ let D_i be a demand bundle of i under the prices $\vec{p_i}$.
- If for all $i \in L$, $D_i = \emptyset$ then terminate.
- For all $i \in L$ with $D_i \neq \emptyset$, let

$$p_i(D_i) \leftarrow p_i(D_i) + \epsilon.$$

3 Other Settings

Below are the classes of valuations that we consider and their abbreviations.

Gen	General (unrestricted) valuations.
SubA	Subadditive valuations, i.e., where $v(S \cup T) \leq v(S) + v(T)$ for all S, T.
XOS	All valuations that can be represented by XOR-of-ORs bids with singleton atomic bundles (see Section 11.4 in AGT by Noam Nisan).
SubM	Submodular valuations, i.e., where for every two bundles S and T we have $v(S) + v(T) \geq v(S \cup T) + v(S \cap T)$.
Subs	(Gross-)substitutes valuations: see definition above.
k**Dup**	Combinatorial auctions with k duplicates of each good. Each bidder desires at most one unit of each good.
Proc	Procurement auctions, where a single buyer needs to buy a set of m items from n suppliers. The suppliers have privately known costs for bundles of items. The buyer aims to minimize the total cost paid.

Figure 1 shows some of the best algorithmic results, incentives compatible approximation results and lower bounds for different classes of combinatorial-auction valuations.

All results apply to a polynomial number of queries of the specified type. Results without references can be trivially derived from other entries in this table. The word "rand" implies that the result is achieved by a randomized algorithm; otherwise, the results correspond to deterministic algorithms only. Results that use ϵ hold for any $\epsilon > 0$. For the simplicity of the presentation, the constants of the asymptotic results are ignored (i.e., we drop the $\mathcal{O}$ and Ω notations).

Class	Queries	Approx	IC approx	Lower bound
Gen	Any	$\sqrt{m}$	$\frac{m}{\sqrt{\log m}}$ $\sqrt{m}$ (rand)	$m^{\frac{1}{2}-\epsilon}$ Section 1.6, [NS06]
	Value	$\frac{m}{\sqrt{\log m}}$	$\frac{m}{\sqrt{\log m}}$[HKDMT04]	$\frac{m}{\log m}$ [BN05a, DS05]
	Demand	$\sqrt{m}$ [BN05a]	$\frac{m}{\sqrt{\log m}}$ $\sqrt{m}$ (rand) [LS05, DNS06]	$m^{\frac{1}{2}-\epsilon}$
SubA	Value	$\sqrt{m}$	$\sqrt{m}$ [DNS05]	$m^{\frac{1}{4}}$
	Demand	2 (rand) [Fei06]	$\sqrt{m}$	2 [DNS05]
XOS	Value	$\sqrt{m}$	$\sqrt{m}$	$m^{\frac{1}{4}}$ [DS06]
	Demand	2 [DNS05] $\frac{e}{e-1}$ (rand) [Fei06]	$\sqrt{m}$ $\log^2 m$ (rand) [DNS06]	$\frac{e}{e-1}$ [DNS05]
SubM	Value	2 [LLN06]	$\sqrt{m}$	$\frac{e}{e-1}$ [KLMM05]
	Demand	2 $\frac{e}{e-1}$-10^{-4} (rand) [FV06]	$\sqrt{m}$ $\log^2 m$ (rand)	$\frac{276}{275}$ [FV06]
Subs	Value	1 [Ber05]	1	
	Demand	1 [GS99, BM97]	1	
k**Dup**	Demand	$m^{\frac{1}{k+1}}$ [BKV05, DS05]	$k \cdot m^{\frac{1}{k-2}}$ [BGN03]	$m^{\frac{1}{k+1}-\epsilon}$ [BGN03, DS05]
Proc	Any	$\ln n$ [NS06]	-	$\log n$ [Nis02]

Figure 1: Existing Algorithmic Results

[NS06]: Nisan and Segal, 2006; [BN05a]: Blumrosen and Nisan, 2005; [DS05]: Dobzinski and Schapira, 2005; [LS05]: Lavi and Swamy, 2005; [DNS061]: Dobzinski et al., 2006; [Fei06]: Feige, 2006; [DNS05]: Dobzinski et al., 2005; [DS06]: Dobzinski and Schapira, 2006; [LLN06]: Lehmann et al., 2006; [KLMM05]: Khot et al., 2005; [FV06]: Feige and Vondrak, 2006; [Ber05]: Bertelsen, 2005; [GS99]: Gul and Stacchetti, 1999; [BM97]:Bikhchandani and Mamer, 1997; [BKV05]: Briest et al., 2005; [BGN03]: Bartal et al., 2003;[Nis02]: Nisan, 2002.